

Breaking the Isolation: Coordinated Multi-agent Fuzzing for Smart Contracts with Multi-dimensional Objective Learning

Ruichao Liang

School of Cyber Science
and Engineering, Wuhan
University, China

liangruichao@whu.edu.cn

Ruochen Cao

School of Cyber Science
and Engineering, Wuhan
University, China

crc2019@whu.edu.cn

Jing Chen

School of Cyber Science
and Engineering, Wuhan
University, China

chenjing@whu.edu.cn

Kun He

School of Cyber Science
and Engineering, Wuhan
University, China

hekun@whu.edu.cn

Yebo Feng

Nanyang Technological University
Singapore

yebo.feng@ntu.edu.sg

Yue Xue

MetaTrust Labs
Singapore

nerbonic@gmail.com

Cong Wu*

School of Cyber Science and Engineering,
Wuhan University, China

cnacwu@whu.edu.cn

Abstract

Smart contracts manage critical assets in decentralized systems but are prone to vulnerabilities. While fuzzing has proven effective in software testing, existing contract fuzzers often rely on ad-hoc optimization strategies with limited objective guidance, typically driven by coverage or simple heuristics. This narrow focus leads to high false negative rates, as important perspectives of smart contracts, like semantic, state, and economic effects, are overlooked during transaction sequence generation. Moreover, coordinating these dimensions into fuzzing is challenging because their divergent objectives and complex interactions make ad-hoc adaptations to fitness functions or mutation strategies largely ineffective.

To fill the gaps, we propose SYNERFUZZ, a multi-agent deep reinforcement learning-based fuzzer. We formulate fuzzing as a Decentralized Partially Observable Markov Decision Process, where each agent is assigned a distinct testing objective extracted from the target contract. Through coordinated learning, DRL agents integrate multi-dimensional signals to construct high-quality sequences, enabling SYNERFUZZ to uncover both subtle semantic flaws and economically profitable behaviors. Compared to 9 SOTA tools, SYNERFUZZ detects up to 4.2× more vulnerabilities while reducing time and transaction overhead by 70.6% and 86.8%. In real-world scenarios, SYNERFUZZ reproduces 33 historical DeFi exploits with over \$40 million losses and discovers 17 0-day vulnerabilities in 4 projects managing over \$179.8 million, earning \$3,800 bug bounties.

CCS Concepts

• Security and privacy → Vulnerability scanners.

Keywords

Smart contract, Vulnerability, Fuzzing, Deep reinforcement learning

ACM Reference Format:

Ruichao Liang, Ruochen Cao, Jing Chen, Kun He, Yebo Feng, Yue Xue, and Cong Wu. 2026. Breaking the Isolation: Coordinated Multi-agent Fuzzing for Smart Contracts with Multi-dimensional Objective Learning. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26)*, October 12–16, 2026, Munich, Germany. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3832783.3837466>

1 Introduction

Smart contracts are self-executing programs on blockchains that enforce terms without intermediaries [3, 30, 31]. However, vulnerabilities in smart contracts have witnessed significant financial losses [25]. Fuzzing is widely used for detecting vulnerabilities by automatically generating inputs to exercise the target program [52].

However, when fuzzing is applied to smart contracts [19, 26, 47, 49], domain-specific characteristics introduce blind spots that limit its effectiveness. Specifically, we identify three key limitations. (i) **State awareness**: the complex contract state space requires exploration from the state perspective; without it, fuzzers miss critical state-dependent vulnerabilities. (ii) **Semantic awareness**: many contract bugs arise from domain-specific logic flaws, especially in DeFi protocols, where correct execution depends on intricate contract semantics; if fuzzers are unaware of such semantics, they tend to generate syntactically valid yet semantically meaningless sequences. (iii) **Economic awareness**: the economic nature of contracts demands attention from the economic perspective through asset transfers; ignoring this causes fuzzers to overlook high-impact, profitable vulnerabilities. Together, these characteristics suggest that effective contract fuzzing should be guided by multiple complementary objectives spanning code, state, semantic, and economic perspectives. Existing work has considered some of these perspectives, such as state [17, 34, 48] or economic factors [23, 51], but to the best of our knowledge, no prior approach unifies all four in contract fuzzing, resulting in a high false negative rate.

Achieving such multi-dimensional fuzzing is non-trivial. Prior studies have shown that adapting fitness functions or mutation strategies can bias fuzzing toward specific objective [2, 35, 45]. However, integrating multiple objectives into transaction sequence construction is substantially more challenging. First, heterogeneous and often competing feedback from multi-dimensional objectives is difficult to unify within a fitness function, making it hard to

* Cong Wu is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. ASE '26, Munich, Germany

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2882-2/2026/10
<https://doi.org/10.1145/3832783.3837466>

select promising transaction sequences. Second, complex and non-linear interactions across dimensions make heuristics or ad-hoc rules insufficient for guiding transaction composition and mutation, preventing fuzzers from progressively converging toward multiple objectives. Finally, the absence of adaptive decision-making mechanisms limits the ability to balance trade-offs among divergent objectives during fuzzing, hindering the evolution of transaction sequences toward vulnerability triggering.

Motivation. We draw inspiration from Decentralized Partially Observable Markov Decision Process (Dec-POMDP), which are designed to handle collaborative decision-making scenarios with three key characteristics: (i) *Partial observability* allows each agent in Dec-POMDP to focus on its domain without being directly influenced by others; (ii) *Decentralized coordination* enables multiple agents to collaborate by learning the interactions and influences across different domains; (iii) *Adaptive decision-making* allows agents to adjust strategies based on evolving campaign dynamics rather than relying on static rules. We therefore reframe contract fuzzing as Dec-POMDP, a collaborative decision-making process rather than single-objective optimization, where each agent corresponds to a specific testing objective and collaborates to achieve the common goal of contract vulnerability discovery.

SYNERFUZZ. We propose SYNERFUZZ, a multi-agent deep reinforcement learning-based fuzzer. We enhance contract fuzzing by incorporating multi-dimensional objectives from *code*, *state*, *semantic*, and *economic* perspectives, modeling the process as a Dec-POMDP to learn from heterogeneous signals and guide adaptive fuzzing strategies. Specifically, we introduce these fuzzing objectives through specialized analysis: pattern matching for critical code identification, backward analysis for state condition extraction, program dependence graph (PDG) analysis for contract semantic recognition, and financial impact assessment for profitability awareness. Each agent in the Dec-POMDP is instantiated using deep reinforcement learning (DRL), working together to learn strategies that optimize function selection, transaction ordering, and parameter mutation to construct effective transaction sequences.

We evaluate SYNERFUZZ against 9 state-of-the-art (SOTA) tools, including 8 fuzzers and one LLM-based exploit generator. Across the newly collected dataset and large-scale stress testing, SYNERFUZZ uncovers up to 4.2× more vulnerabilities while reducing execution time and transaction overhead by up to 70.6% and 86.8%, respectively. Notably, it uncovers 693 vulnerabilities that all SOTA tools fail to detect with superior code coverage. In real-world testing, SYNERFUZZ reproduces 33 historical DeFi exploits, recovering \$22.1M in profit, and uncovers 17 0-day vulnerabilities in 4 projects managing over \$179.8 million USD, earning \$3,800 bounty rewards.

Contributions. We make the following contributions:

- We demystify key gaps of current contract fuzzers: i) insufficient awareness of contract-specific characteristics, including state evolution, DeFi semantics, and economic effects during transaction sequence construction, leads to ineffective exploration and high false-negative rates, and ii) ad-hoc adaptations to fuzzing strategies fail to coordinate these multiple testing objectives.
- We introduce a multi-dimensional objective formulation for transaction sequence construction, realized through specialized analyses and a Dec-POMDP-based fuzzing framework, enabling

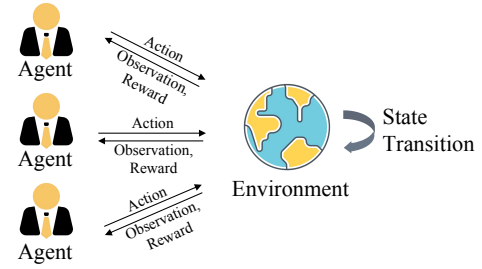


Figure 1: Illustration of the Dec-POMDP model.

DRL agents to learn strategies that generate semantically meaningful and economically impactful transaction sequences.

- We propose SYNERFUZZ, a multi-agent DRL-based fuzzer. Evaluation against nine tools shows its superior effectiveness in exploiting both subtle semantic flaws and economically profitable behaviors. It also shows real-world effectiveness by reproducing historical exploits and discovering 0-day vulnerabilities.

2 Background

2.1 Smart Contract Vulnerability

Smart contract vulnerabilities differ from traditional software vulnerabilities in three aspects. (i) They often emerge in specific states. Because smart contracts maintain persistent on-chain state, many vulnerabilities are only exploitable under specific contract states [7], requiring carefully constructed transaction sequences to reach them. (ii) They are primarily logical flaws. Running in the Ethereum Virtual Machine (EVM) [46], smart contracts are less prone to low-level memory errors and more vulnerable to logical inconsistencies in business rules, such as reentrancy and access control bugs. (iii) They often lead to financial losses. As asset-managing programs, contracts can be exploited to steal funds, making financial damage a critical consideration that existing approaches often overlook.

2.2 Dec-POMDP

Dec-POMDP is a standard model for decentralized multi-agent decision-making under partial observability [6]. It generalizes MDPs and POMDPs from a single agent to multiple agents interacting with a shared environment. As shown in Figure 1, in a Dec-POMDP, each agent chooses actions based on local observations, while the environment transitions according to the joint actions of all agents and produces corresponding observations and rewards. Formally, a Dec-POMDP is defined as $\langle \mathcal{I}, \mathcal{S}, \{A_i\}, \{O_i\}, \mathcal{T}, \{R_i\}, \{Z_i\}, \gamma \rangle$, where $\mathcal{I}$, $\mathcal{S}$, A_i , and O_i denote the agent set, state space, and the action and observation spaces of agent i , respectively; $\mathcal{T}$ is the transition function, R_i is the reward function, Z_i is the observation function, and γ is the discount factor.

3 Motivation and Problem Formulation

3.1 Motivating Example

Consider the EtherVault in Listing 1. Users can deposit funds, obtain VIP status, and withdraw once the accumulated fees exceed a threshold. Although `withdrawAll` is guarded by `noReentrant`,

```

1 contract EtherVault {
2   // ...
3   function deposit() public payable {
4     require(msg.value > 1 ether, "Insufficient");
5     fee += 1 ether;
6     balances[msg.sender] += msg.value - 1 ether; }
7   function setVIP() public {
8     require(!isVIP[msg.sender], "Already VIP");
9     require(balances[msg.sender] >= VIPfee);
10    balances[msg.sender] -= VIPfee;
11    isVIP[msg.sender] = true; }
12   function withdrawAll() public noReentrant{
13     require(fee >= withdrawThreshold);
14     uint256 amount = balances[msg.sender];
15     (bool success,) = msg.sender.call{value: amount};
16     require(success, "Transfer failed");
17    balances[msg.sender] = 0; }

```

Listing 1: Vulnerable snippet of EtherVault.

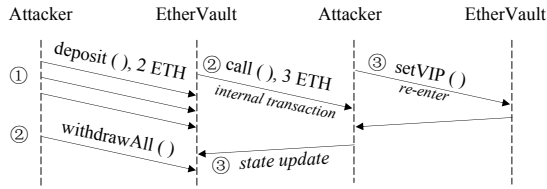


Figure 2: Exploitation vectors of reentrancy in EtherVault.

the contract remains vulnerable to cross-function reentrancy. As illustrated in Figure 2, an attacker first invokes `deposit` repeatedly to increase both their balance and the global fee, and then exploits the external call in `withdrawAll` to reenter the contract through `setVIP`. By doing so during a transient inconsistent state, where the balance still satisfies the VIP requirement but is about to be reset, the attacker can bypass the VIP fee and withdraw all funds. Existing fuzzers often miss such an exploit not because of insufficient coverage, but because they cannot effectively reason across multiple perspectives.

(i) *Exploitation requires state, semantic, and economic awareness.* Coverage alone is insufficient, because the bug is only exposed under a specific state configuration, including the fee threshold, the attacker’s balance, and the VIP status. Moreover, the exploit depends on a semantic pattern: cross-function reentrancy through `setVIP` during `withdrawAll`, that cannot be captured by isolated function execution. It also has clear economic significance, as the attacker gains privileged access while avoiding the intended VIP fee. Fuzzers that ignore these state, semantic, and economic signals are unlikely to distinguish the exploit from benign executions.

(ii) *Multi-dimensional objectives are difficult to coordinate.* State, semantic, and economic objectives differ in scale, representation, and mutation sensitivity, and their interactions are often non-linear. As a result, simple static combinations of such objectives can misguide exploration rather than promote it. Their relative importance may also shift during fuzzing, requiring adaptive coordination instead of fixed fitness formulations.

We further tested EtherVault using Foundry [18] invariant-testing setup with runs=512, depth=128, call_override=true, and handlers covering the relevant public functions. Across 10 independent runs, Foundry did not trigger the vulnerability. This example highlights the limitations of existing fuzzers and motivates a new

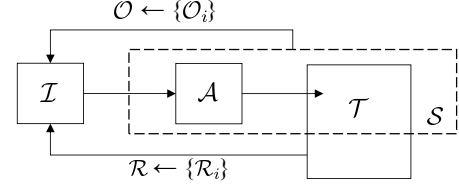


Figure 3: Formulation of contract fuzzing as a Dec-POMDP.

approach that can coordinate multi-dimensional objectives to guide exploration toward real-world complex vulnerabilities.

3.2 Formulating Fuzzing as Dec-POMDP

Figure 3 illustrates our Dec-POMDP formulation of contract fuzzing. Let $\mathcal{I}$ denote the set of DRL agents, each corresponding to one fuzzing objective dimension. Each agent interacts with the environment by choosing actions from a predefined action space $\mathcal{A}$, whose detailed design is presented in Section 4. In our genetic-algorithm-based framework, $\mathcal{A}$ consists of three genetic operators:

$$\mathcal{A} = \mathcal{A}_{\text{sel}} \times \mathcal{A}_{\text{cross}} \times \mathcal{A}_{\text{mut}}, \quad (1)$$

where $\mathcal{A}_{\text{sel}}$, $\mathcal{A}_{\text{cross}}$, and $\mathcal{A}_{\text{mut}}$ represent the selection, crossover, and mutation operators. State space $\mathcal{S}$ captures the fuzzing environment state, including contract source code, internal contract state, input transactions along with their execution traces, coverage information, and any detected vulnerabilities. Since each agent specializes in different fuzzing objective, the observation space $\mathcal{O} \subseteq \mathcal{S}$ defines the specific fuzzing environment visible to each agent. The reward function R provides specialized feedback based on different fuzzing objectives. In this formulation, R and $\mathcal{O}$ are agent-specific, i.e.,

$$R \leftarrow \{R_1, R_2, \dots, R_n\}, \quad \mathcal{O} \leftarrow \{O_1, O_2, \dots, O_n\}. \quad (2)$$

The fuzzing backend is primarily implemented through an instrumented EVM and modeled by the state transition function $\mathcal{T}$. It takes the current state s and an action a as input, and executes the contract to produce a new state s' along with a reward r :

$$\begin{aligned} s' &= \mathcal{T}(s, a), \quad a \in \mathcal{A}, \quad s, s' \in \mathcal{S}, \\ r &= R_i(\mathcal{T}(s, a)), \quad R_i \in R, \quad r \in \mathbb{R}. \end{aligned} \quad (3)$$

Each agent $i \in \mathcal{I}$ proposes a candidate action $a_i \in \mathcal{A}$, which is then aggregated through a joint decision function f to produce the final action:

$$a = f(a_1, a_2, \dots, a_n), \quad (4)$$

Each action a_i is sampled from the agent’s local strategy:

$$a_i \sim \pi_i(a \mid O_i), \quad \pi_i(a \mid O_i) \in [0, 1], \quad O_i \in \mathcal{O}, \quad (5)$$

The global strategy combines all specialized strategies:

$$\pi_{\text{joint}}(a \mid \mathcal{O}) = f(\pi_1(a \mid O_1), \dots, \pi_n(a \mid O_n)). \quad (6)$$

The training objective is to learn an optimal fuzzing strategy π_i^* for each agent $i \in \mathcal{I}$, maximizing its expected cumulative reward for effective vulnerability detection:

$$\pi_i^* = \arg \max_{\pi_i} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_i(\mathcal{T}(s_t, a_t)) \right], \quad (7)$$

where $\gamma \in [0, 1)$ is the discount factor, and the expectation reflects the average outcome when the agent follows its strategy in the environment. Through this formulation, we transform fuzzing into

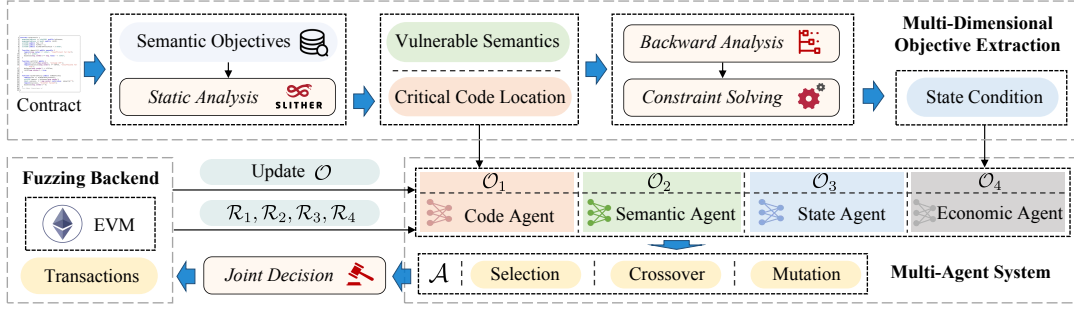


Figure 4: Workflow of SYNERFUZZ.

an adaptive, collaborative process where agents work together to generate effective sequences and expose contract vulnerabilities.

4 SYNERFUZZ

4.1 Overview

As shown in Figure 4, SYNERFUZZ consists of three phases: objective extraction, multi-agent system, and fuzzing backend. First, the contract undergoes a multi-step analysis to extract semantic, code, and state objectives. Semantic objectives capture vulnerability-relevant behaviors from IR-level relations such as control and data dependencies. Code objectives identify critical control-flow locations related to these behaviors. State objectives characterize the conditions required to trigger them. The multi-agent system contains four DRL agents, each operating over its own observation space O_n :

- *Code agent* directs testing toward critical code locations with high vulnerability potential.
- *State agent* guides the contract toward specific states essential to vulnerability triggering.
- *Semantic agent* ensures test cases exhibit behaviors consistent with vulnerability-relevant contract semantics, guiding transaction sequences toward semantically meaningful interactions.
- *Economic agent* focuses on behaviors that reveal financially exploitable vulnerabilities with concrete profit potential.

All agents act over a shared action space A defined by genetic operators, including selection, crossover, and mutation. Their actions are aggregated to construct transaction sequences, which are executed by the fuzzing backend. Based on extracted objectives and runtime feedback, each agent receives a reward R_n and gradually learns to coordinate with others to improve vulnerability discovery.

4.2 Multi-Dimensional Objective Extraction

Semantic objective library. As shown in Figure 5, we maintain a semantic objective library organized as a three-layer hierarchy:

$$\mathcal{L} : \sigma \mapsto v \mapsto p, \quad p = \langle id, P, D, \ell \rangle.$$

Here, σ denotes a contract-semantic branch, such as AMM, lending, or vault. Each branch is further refined into vulnerability categories v , and each category is instantiated as a leaf semantic objective p . In addition to domain-specific branches, the first layer also includes a General branch for cross-cutting contract behaviors that are not tied to a specific protocol role. Each leaf objective consists of a PDG pattern P , its Slither-IR detector implementation D , and a critical

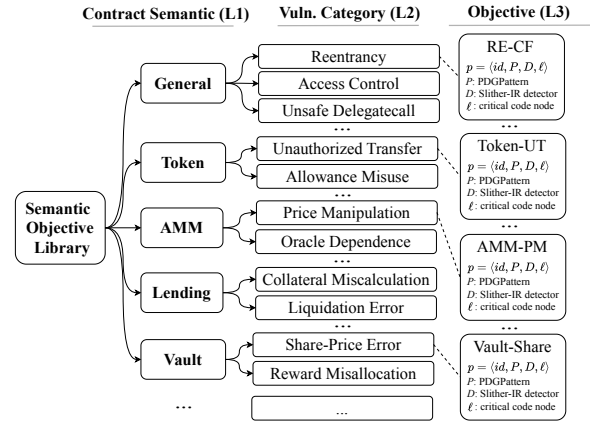


Figure 5: Semantic objective library schema.

node ℓ marked in the pattern. A PDG pattern is an abstract control-/data-flow graph that captures the recurring execution signature of a vulnerability root cause, such as an external call before a state update, or a user-controlled delegatecall target.

The library is manually constructed by domain experts under a systematic taxonomy-to-pattern procedure. We collect recurring contract vulnerabilities from public taxonomies [44] and audit reports [9, 43]. These sources are first organized by contract semantic and vulnerability category. For each recurring root cause, we abstract its common control-/data-flow signature into a PDG pattern, mark the critical node in the pattern, and implement it as a Slither-IR detector. Before inclusion, the detector is calibrated on the source corpus and historical cases to ensure that it recalls the intended root-cause pattern. Semantic library is designed to produce candidate testing objectives, rather than vulnerability reports. Therefore, when designing PDG patterns and their detectors, we follow a recall-oriented approximate-matching principle to expose sufficient vulnerability-prone behaviors, while final alarms are emitted only after concrete executions are validated by runtime oracles. New objectives can be added by instantiating a new leaf pattern and detector through the same procedure.

Identification of vulnerability-prone semantics and code locations. Given a target contract, SYNERFUZZ first builds its PDG using SLITHER [16]. For each selected leaf objective $p = \langle id, P, D, \ell \rangle$, SYNERFUZZ runs specific detectors D to match the PDG patterns P against the target PDG. A successful match produces a concrete

Algorithm 1: State Condition Identification

```

1 Procedure PATHRESOLVER( $c, l$ ):
   Input :Contract  $c$ , Code Location  $l$ 
   Output:State Condition  $sc$ 
2    $constraint \leftarrow \emptyset$ 
3   for  $p$  in ALLPATH( $c, l$ ) do
4     for  $bp$  in ALLBRANCHPOINT( $p, l$ ) do
5        $bp.constraint \leftarrow \text{BACKWARDANALYSIS}(p, bp)$ 
6        $p.constraint \leftarrow p.constraint \wedge bp.constraint$ 
7      $constraint \leftarrow constraint \vee p.constraint$ 
8    $sc \leftarrow \text{SOLVE}(constraint)$ 
9 Procedure BACKWARDANALYSIS( $p, bp$ ):
   Input :Path  $c$ , Branch Point  $bp$ 
   Output:Branch Point constraint  $bp.constraint$ 
10   $bp.constraint, target \leftarrow \text{INIT}(p, bp)$ 
11   $instructions \leftarrow \text{PRECEDINGINS}(p, bp)$ 
12   $evmStack \leftarrow \emptyset$ 
13  while  $instructions \neq \emptyset$  do
14     $ins \leftarrow instructions.pop()$ 
15     $evmStack \leftarrow \text{REBUILD}(ins, evmStack)$ 
16     $bp.constraint, target \leftarrow \text{UPDATE}(evmStack, ins, target)$ 
17   $bp.constraint \leftarrow \text{FILTERING}(bp.constraint)$ 

```

pattern instance $\mu(P)$, which serves as the vulnerability-prone semantic objective for the semantic agent. The corresponding code target is obtained directly as $\mu(\ell)$, because ℓ is the critical node marked in P . SYNERFUZZ then maps it to corresponding bytecode basic block via source maps and uses it as the critical code location for the code agent. During matching, SYNERFUZZ includes the General branch by default and selects domain-specific branches either from user-specified semantics or by deterministic keyword matching. Each semantic branch maintains a keyword set, which is matched against contract names, ABI entries, and function signatures.

Identification of state conditions. We design a state condition identification algorithm (Algorithm 1) to extract contract states required for reaching vulnerability-relevant code locations. The algorithm consists of two procedures: PATHRESOLVER and BACKWARDANALYSIS. Given a target contract and a critical code location, PATHRESOLVER first enumerates all execution paths from the entry to the target location. For each path, it identifies all branch points and invokes BACKWARDANALYSIS to derive the state constraints required to follow each branch. The constraints of branch points along the same path are conjuncted, while constraints from different paths are disjuncted, yielding a path-complete constraint formula for reaching the target location. We then solve the formula with Z3 SMT solver to obtain concrete state conditions, represented as value ranges of relevant state variables, e.g., $\{v_1 \in [\min_1, \max_1], v_2 \in [\min_2, \max_2], \dots\}$. Solving is performed with a 30-second timeout. If the formula is unsatisfiable or solving times out, the corresponding state objective is discarded. For each branch point, BACKWARDANALYSIS starts from the branch condition implied by the jump direction and performs a backward scan over the preceding bytecode instructions. During this process, it reconstructs the EVM stack and incrementally updates the constraint by tracing how branch-related variables are introduced, computed, or loaded from storage. Finally, it filters the resulting constraint to retain only predicates over contract state variables.

Note that the extracted state conditions serve only as preliminary objectives rather than confirmed vulnerabilities. Because the semantic objective library is heuristic and the static analysis is path-insensitive and over-approximate, which may contain false

positives [33]. Therefore, subsequent fuzzing is still required to validate whether these candidate behaviors are truly exploitable.

4.3 Multi-Agent System

4.3.1 Observation Space and Reward. The agent system is based on Deep Q-Network (DQN) [32]. As shown in Table 1, agent's observation space is designed to contain sufficient information for decision-making. The reward function reflects domain-specific objectives and is computed as a bounded delta between generations to ensure stability.

Code agent directs fuzzing toward potentially vulnerable code locations within the contract. Its observation space incorporates critical code locations L identified in objective extraction phase, alongside a code distance metric $D(T)$ that quantifies proximity between transaction executions and these critical locations. Let $d(i, j)$ denote the shortest path distance between basic blocks i and j . For a basic block b , its distance to all targets in L is defined as:

$$D(b) = \begin{cases} 0, & \text{if } \exists l \in L : d(b, l) = 0, \\ |L| \times \left[\sum_{l \in L} d(b, l)^{-1} \right]^{-1}, & \text{otherwise.} \end{cases} \quad (8)$$

This distance equals zero if block b directly connects to any critical location; otherwise, it equals the harmonic mean of distances to all critical locations. For each transaction T executing a sequence of basic blocks, we compute $D(T)$ by averaging the distances of the five closest blocks B to critical code locations:

$$D(T) = \frac{1}{5} \times \sum_{b \in B} D(b). \quad (9)$$

In each generation, population P comprises multiple individuals, each representing a transaction sequence. For an individual I , the code distance is the minimum over its execution sequence:

$$D(I) = \min_{T \in I} D(T). \quad (10)$$

The population's code distance is average of the individual's:

$$D(P) = \frac{1}{|P|} \sum_{I \in P} D(I). \quad (11)$$

The code agent's reward is calculated as the bounded difference in population code distance between current and previous generations:

$$R_{\text{code}} = \max(-5, \min(5, D(P_{\text{current}}) - D(P_{\text{prev}}))). \quad (12)$$

State agent guides fuzzing within the contract state space to satisfy vulnerability-triggering conditions. Its observation features include state conditions sc and a state distance metric $D_s(T)$ that quantifies how closely transactions manipulate contract state to meet these conditions. Let $d_s(i, j)$ represent the minimum numerical distance between a value i and a condition range j . For a specific condition $c \in sc$, the state distance for transaction T is defined as:

$$D_s(c) = \sum_{v \in V} d_s(v, c), \quad (13)$$

where V is the state variables involved in condition c . The overall state distance of transaction T across all conditions is:

$$D_s(T) = \begin{cases} 0, & \text{if } \exists c \in sc : D_s(c) = 0, \\ |sc| \times \left[\sum_{c \in sc} D_s(c)^{-1} \right]^{-1}, & \text{otherwise.} \end{cases} \quad (14)$$

For each individual I , we calculate the state distance as the minimum across all transactions in its execution sequence:

$$D_s(I) = \min_{T \in I} D_s(T). \quad (15)$$

Table 1: Observation space of deep reinforcement learning agents.

Observation Feature	Source	Description	Agent
Critical Code Location: L Code Distance: $D(T)$	Objective Extraction Fuzzing Backend	Contract control flow graph (CFG) with critical code locations: $\{l_1, l_2, \dots\}$ Harmonic mean of basic block distances between transactions' execution path and critical code locations	Code Agent Code Agent
State Condition: sc State Distance: $D_s(T)$	Objective Extraction Fuzzing Backend	Set of conditions regarding state variables: $\{c_1, c_2, \dots\}$ Harmonic mean of value distances between transactions' state variable and state condition	State Agent State Agent
Vulnerable Semantics: A Semantic Distance: $D_{sm}(I)$	Objective Extraction Fuzzing Backend	Set of program dependency graph (PDG) with vulnerable semantics: $\{a_1, a_2, \dots\}$ Harmonic mean of graph edit distances between individuals' execution path and vulnerable path	Semantic Agent Semantic Agent
Value Transfer Events Attacker-State Changes Code Coverage Opcode Frequencies	Fuzzing Backend Fuzzing Backend Fuzzing Backend Fuzzing Backend	External calls with value transfer Increases in mappings where attacker is the key Current code coverage percentage Execution counts of key opcodes (CALL, SLOAD, SSTORE, etc.)	Economic Agent Economic Agent Economic Agent Economic Agent
Last Population Mutation Pool Historical Population	Fuzzing Backend Fuzzing Backend Fuzzing Backend	Seed sequences from the previous generation Mutation candidate values Seed sequences and execution paths from historical populations	All Agents All Agents All Agents

The population's overall state distance is computed as the average of all individual distances:

$$D_s(P) = \frac{1}{|P|} \sum_{I \in P} D_s(I). \quad (16)$$

The state agent's reward comprises two components: a distance improvement measure and a target achievement bonus:

$$R_{\text{state}} = R_{\text{distance}} + R_{\text{target}}, \quad (17)$$

$$R_{\text{distance}} = \max(-5, \min(5, D_s(P_{\text{current}}) - D_s(P_{\text{prev}}))), \quad (18)$$

$$R_{\text{target}} = \min(1, 0.2 \times n), \quad (19)$$

where n is the number of state variables within the target.

Semantic agent guides fuzzing toward triggering vulnerability-related behaviors. It leverages program dependency graphs (PDGs) to capture semantic relationships between program elements. Its observation space includes vulnerable semantics A and semantic distance metrics $D_{sm}(I)$ that quantify similarity between execution paths and these patterns. For each individual, we generate its corresponding PDG G and define the semantic distance $D_{sm}(I)$ as the harmonic mean of windowed graph edit distances (WGED) between G and each vulnerable pattern in A :

$$D_{sm}(I) = |A| \times \left(\sum_{a \in A} \text{WGED}(G, a)^{-1} \right)^{-1}. \quad (20)$$

We use WGED instead of standard graph edit distance to mitigate the impact of size disparities between graphs, enabling more accurate similarity assessment. The population's semantic distance is computed as the average of individual distances:

$$D_{sm}(P) = \frac{1}{|P|} \sum_{I \in P} D_{sm}(I). \quad (21)$$

Semantic agent's reward is the bounded difference in population semantic distance between current and previous generations:

$$R_{\text{semantic}} = \max(-5, \min(5, D_{sm}(P_{\text{current}}) - D_{sm}(P_{\text{prev}}))). \quad (22)$$

Economic agent focuses on attacker-centric profits during fuzzing, while also considering general fuzzing metrics. In Table 1, its observation space incorporates value transfers, state changes favoring the attacker, code coverage, and execution frequency of critical instructions. Like all agents, it also observes transactions from previous generations, the current mutation pool, and the historical population, enabling comprehensive awareness of the fuzzing environment. The economic agent reward is:

$$R_{\text{economic}} = R_{\text{profit}} + R_{\text{coverage}} + R_{\text{vulnerability}}, \quad (23)$$

$$R_{\text{profit}} = \min(5, 0.1 \times N_{\text{ether}} + 0.05 \times N_{\text{asset}}), \quad (24)$$

where N_{ether} counts external calls transferring ether to the attacker, and N_{asset} tracks increases in contract state mappings with the attacker's address as key. This reward component is capped at +5 per generation to maintain balanced optimization. $R_{\text{coverage}} +1$ when code coverage increases and -1 after 10 generations without improvement, and $R_{\text{vulnerability}} +10$ for each vulnerability discovered.

4.3.2 Action Space. Agents share a unified action space with three genetic operators. The *selection operator* selects two parent indices (i, j) from the previous generation. The *crossover operator* determines how transaction sequences from parents are combined. Agents select from six strategies that concatenate father (F) and mother (M) sequences in different patterns. The *mutation operator* specifies which transaction parameters to modify and selects replacement values from the mutation pool. Each agent learns strategies and makes decisions based on its unique observations and rewards. To integrate decisions from four specialized agents and determine the optimal action a^* , we propose a joint decision function using confidence-weighted preference aggregation:

$$a^* = \arg \max_{a \in A} \sum_{i=1}^4 c_i(o_i, a) \cdot \text{pref}_i(a|o_i), \quad (25)$$

where a is a candidate action, $c_i(o_i, a)$ denotes the confidence of agent i on action a , and $\text{pref}_i(a|o_i)$ denotes its action preference. Both terms are derived from Q -values:

$$\text{pref}_i(a|o_i) = \frac{\exp(Q_i(o_i, a)/\tau)}{\sum_{a' \in A} \exp(Q_i(o_i, a')/\tau)}, \quad (26)$$

$$c_i(o_i, a) = \frac{Q_i(o_i, a) - \min_{a' \in A} Q_i(o_i, a')}{\max_{a' \in A} Q_i(o_i, a') - \min_{a' \in A} Q_i(o_i, a') + \epsilon}. \quad (27)$$

Here, pref_i is a softmax-normalized action preference, and c_i is a normalized action-wise Q -advantage. This joint decision function balances each agent's objective-specific preference and Q -value-derived confidence when determining the collective action. Since all candidate actions are scored under the same aggregation rule, divergent agent preferences are naturally absorbed into the final score. Thus, conflicts are handled by soft arbitration rather than requiring agent agreement.

4.3.3 Network of Agents. Figure 6 illustrates our DQN-based agent architecture. Each agent takes the current observation s as input, outputs a discrete action, and stores the transition (s, a, r, s') in a

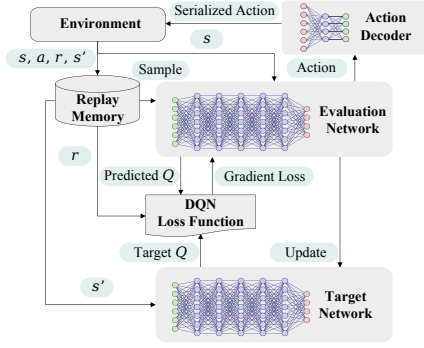


Figure 6: Architecture of agent neural network.

replay buffer. The evaluation network is trained on sampled transitions using the standard DQN loss, while the target network provides stable target Q -values and is updated periodically. We further extend DQN with two components. An *Action Decoder* converts network outputs into structured fuzzing actions. For joint decision-making, each agent also derives a preference $\text{pref}_i(a \mid o_i)$ and a confidence $c_i(o_i, a)$ from its Q -values, so that action selection reflects both relative preference and decision certainty.

4.4 Fuzzing Backend

Genetic algorithm-based seed formulation. We formulate seed generation as a genetic algorithm over transaction sequences. Each transaction is encoded as a chromosome, whose genes include the function signature, arguments, attached value, block number, and timestamp, together with fixed contextual fields such as sender and receiver addresses. A transaction sequence forms an individual, and individuals constitute the population of each generation. The population is evolved through selection, crossover, and mutation, where mutation draws from random values, incremental adjustments, and previously observed runtime values. The population size is set to twice the number of publicly callable functions.

Cross-contract fuzzing. SYNERFUZZ maintains an address-indexed cache of contract bytecode and storage to support cross-contract execution. When an external call is issued, the VM resolves the callee and switches to the corresponding execution context. For deployed contracts, missing bytecode and uncached storage slots are fetched on demand at the target block height via *Alchemy* APIs [1].

Vulnerability oracles. To detect profit-driven exploits and non-profitable semantic violations, SYNERFUZZ instruments the EVM to collect fine-grained runtime information, including execution traces, state updates, and value transfers. These signals enable financial oracles that identify end-to-end arbitrage behavior, as well as behavior-based oracles that capture violations of fairness, safety properties, or intended contract logic.

4.5 Implementation

Architecture. We implement SYNERFUZZ in about 10K lines of Python. SLITHER [16] is used to translate source code into IR, while backward analysis is built from scratch to extract state conditions and critical code locations. The multi-agent environment is built on *PettingZoo* [15] with customized observations, actions, and rewards, and the DRL agents are implemented using *Tianshou* [42].

For execution, we instrument *py-evm* [14] and integrate it with CONFUZZIUS [17] as the backbone of our fuzzing loop.

Semantic objective library. The library was constructed offline using public vulnerability taxonomies [44] and historical audit reports [9]. We instantiated these sources into structured semantic objectives and excluded entries that cannot be represented as PDG-level control-/data-flow patterns. The current implementation contains 7 semantic branches: General, token, AMM/oracle, lending, vault/staking, bridge, and governance, and further instantiate these branches into 41 vulnerability categories, each represented as one detector-backed PDG pattern. Adding a new objective requires adding a new leaf pattern and detector, without modifying the DRL agents or fuzzing backend.

Learning setup. Training is conducted on a Ubuntu 20.04 server with a 16-core Intel Xeon Gold 6133 CPU and an NVIDIA RTX 4090 GPU. Each agent uses DQN with a 3-layer MLP (128–64–64), and ϵ -greedy exploration. For stability, we use dense delta-based rewards with clipping at each fuzzing iteration. The joint decision function is fixed and applied only during execution.

5 Evaluation

5.1 Evaluation Setups

Research questions. *RQ1:* Does SYNERFUZZ effectively detect more vulnerabilities than SOTA fuzzers? *RQ2:* Does SYNERFUZZ detect vulnerabilities more efficiently than SOTA fuzzers? *RQ3:* Has the DRL framework truly learned effective strategies, and do these strategies meaningfully guide the vulnerability detection? *RQ4:* How does each test objective dimension contribute to SYNERFUZZ’s vulnerability detection capability? *RQ5:* Can SYNERFUZZ detect profitable vulnerabilities that lead to financial losses in production environments, and can it find previously unknown vulnerabilities?

Benchmarks and baselines. We evaluate SYNERFUZZ on four benchmarks (Table 2). **B1** is a curated and deduplicated dataset with fine-grained labels, constructed from prior studies [11, 29, 39, 50, 53]. It covers four common vulnerability categories, with two representative types per category: Access Control Issue, including *Unauthorized Transfer* (UT) and *Unauthorized Self-Destruct* (US); State Inconsistency, including *Intra-Function* and *Cross-Function Reentrancy* (RE); Financial Logic Flaw, including *Price Manipulation* (PM) and *Ether Lock* (EL); and Unsafe External Calls, including *Controlled Delegatecall* (CD) and *Dangerous Delegatecall* (DD). **B2** contains 42,247 unlabeled real-world Ethereum contracts [13] for stress testing. **B3** includes 36 exploited DeFi projects [10] for evaluating profitable-exploit reproduction. **B4** consists of active bug bounty programs from *Secure3* [36] for assessing 0-day vulnerability discovery. For B1 and B2, we compare SYNERFUZZ with seven SOTA fuzzers [8, 17, 29, 34, 38, 40, 51] under standardized behavior-based oracles. For B3, we compare against VERITE [23] and A1 [20], both of which target profitable vulnerabilities.

Experimental implementation. We use 5-fold cross-validation on B1, where each round trains on four folds and evaluates on the remaining held-out fold, and the B1 results aggregate the five held-out evaluations. We train the final policy on the full B1 and evaluate it on B2–B4. Each contract is assigned a testing budget of 5–10 minutes according to code length, using 800 LoC as the cutoff, and the run terminates early once a vulnerability is detected. The

Table 2: Benchmarks overview.

ID	Description	Code Metrics	Size Distribution
B1	1,421 vulnerable smart contracts	Total: 945,836 LoC Avg: 665 LoC	<800 LoC: 1,023, Avg: 286 LoC ≥800 LoC: 398, Avg: 1,679 LoC
B2	42,247 in-the-wild smart contracts	Total: 15,089,799 LoC Avg: 357 LoC	<800 LoC: 38,910, Avg: 262 LoC ≥800 LoC: 3,337, Avg: 1,467 LoC
B3	36 real-world projects with historical exploits and financial losses		
B4	Smart contract projects from Secure3 bug bounty programs		

Table 3: Vulnerability detection results on B1 & B2.

Tool	Benchmark B1			Benchmark B2		
	Vulns	Det.Rt	FNR	Vulns	Det.Rt	FPR
IR-Fuzz [29]	644	45%	55%	644	1.5%	20%
MIDAS [51]	798	56%	44%	1,289	3.1%	21%
ConFuzzius [17]	753	53%	47%	1,079	2.6%	33%
ItyFuzz [38]	959	68%	32%	1,010	2.4%	37%
sFuzz [34]	873	61%	39%	862	2.0%	32%
SMARTIAN [8]	953	67%	33%	574	1.4%	20%
RLF [40]	622	44%	56%	945	2.2%	29%
SYNERFUZZ (Ours)	1,187	84%	16±3.2%	2,381	5.6%	15%

* Vulns denotes the number of detected vulnerabilities, Det.Rt denotes the detection rate, and FNR / FPR denotes the false negative / positive rate.

Table 4: Results on B1 breakdown by vulnerability categories.

Tool	UT	CD	DD	EL	RE	US	PM	Total
IR-Fuzz [29]	30	26	36	60	459	21	12	644
MIDAS [51]	83	15	25	106	501	50	18	798
ConFuzzius [17]	77	35	62	78	420	52	29	753
ItyFuzz [38]	82	17	35	98	652	45	30	959
sFuzz [34]	55	37	57	71	613	27	13	873
SMARTIAN [8]	71	29	62	90	641	50	10	953
RLF [40]	72	43	64	110	263	50	20	622
SYNERFUZZ (Ours)	79	39	69	123	785	53	39	1,187
Uniq.	2	0	2	4	15	2	7	32

Uniq. denotes vulnerabilities detected by SYNERFUZZ but missed by all baselines.

DRL models are trained and executed locally, and training takes about 200 GPU-hours, and B1+B2 testing consumes about 1,200 CPU-hours. All baselines use identical initial seed corpora and time budgets. All split files, random seeds, and benchmark configurations are released with the artifact.

5.2 RQ1: Effectiveness

Overall results. We evaluate SYNERFUZZ and other fuzzers on benchmarks B1 and B2, and the results are presented in Table 3. On B1, SYNERFUZZ identified 1,187 vulnerable contracts with a 16% false negative rate with $\pm 3.2\%$ variance across the five held-out folds, surpassing existing tools by detecting 228 (24%) to 565 (91%) more vulnerabilities, including 32 uniquely identified cases missed by all others. Table 4 shows detailed detection results breakdown by vulnerability categories. SYNERFUZZ detected the highest number of vulnerabilities in 5 out of 7 vulnerability types. On B2, SYNERFUZZ identified 2,381 vulnerabilities in 5.6% of contracts, which is $2.0\times$ to $4.2\times$ more detections than existing tools, including 693 uniquely discovered vulnerabilities. To assess the false positive rate on this unlabeled dataset, we manually verified a random sample of

Table 5: Breakdown of false negatives and false positives

Stage / Root cause		#	Main types
FN (B1)	Library matching: non-canonical price source	2	PM
	Library matching: non-canonical reentry carrier	25	RE
	Library matching: non-standard asset-flow carrier	7	UT
	Fuzzing: deep sequence/path exploration	200	RE, CD/DD
FP (B2)	Runtime oracle: intended privileged behavior	23	UT, US
	Runtime oracle: guarded or non-exploitable behavior	68	RE, CD/DD, EL
	Runtime oracle: incomplete environment modeling	5	UT, PM

Table 6: Results on B2 breakdown by vulnerability categories.

Tool / Metric		UT	CD	DD	RE	EL	US	PM	Total
RLF [40]	#	566	7	11	116	98	110	37	945
	TPR	77%	86%	64%	67%	17%	90%	67%	64%
SMARTIAN [8]	#	387	2	8	17	59	93	8	574
	TPR	87%	100%	88%	88%	13%	93%	75%	70%
sFuzz [34]	#	401	0	3	273	92	88	5	862
	TPR	80%	N/A	100%	60%	20%	83%	60%	62%
ItyFuzz	#	822	5	0	86	30	22	45	1,010
	TPR	63%	60%	N/A	63%	17%	91%	90%	63%
ConFuzzius [17]	#	359	11	7	411	124	133	34	1,079
	TPR	80%	91%	57%	67%	23%	70%	70%	64%
MIDAS [51]	#	950	0	20	169	62	48	40	1,289
	TPR	83%	N/A	95%	77%	13%	97%	63%	70%
IR-Fuzz [29]	#	395	1	23	76	25	101	23	644
	TPR	90%	0%	96%	73%	12%	87%	87%	72%
SYNERFUZZ (Ours)	#	1,487	21	16	534	12	233	78	2,381
	TPR Uniq.	90% 388	90% 9	88% 3	70% 167	92% 2	83% 82	93% 42	86% 693

150 contracts per vulnerability type. Each contract underwent cross-validation by two security experts, with disagreements resolved through team discussion. SYNERFUZZ achieved the lowest false positive rate (15%). Table 6 presents detailed results, showing that UT was the most prevalent vulnerability, while delegatcall-related vulnerabilities were the least common.

False negative & positive analysis. As shown in Table 5, false negatives in B1 arise from two stages. First, *library-matching misses* occur when the vulnerability semantics are covered by the library, but a concrete implementation variant deviates from the current PDG pattern and thus no semantic objective is extracted. These cases include non-canonical price sources in PM vulnerabilities, non-canonical reentry carriers such as modifiers or token callbacks in RE vulnerabilities, and non-standard asset-flow carriers in UT vulnerabilities. Second, *fuzzing-stage misses* occur when objectives are extracted but SYNERFUZZ fails to trigger the vulnerability within the time budget. This is the dominant source of B1 false negatives: 200 cases are due to deep sequence or path exploration, mainly involving RE and CD/DD, where cross-function combinations grow rapidly and vulnerable sites may lie deep in execution paths.

False positives in B2 mainly originate from runtime oracles. The semantic library generates candidate objectives; a report is produced only when fuzzing reaches an execution that satisfies the corresponding oracle. As shown in Table 5, false positives arise from three oracle-level causes: intended privileged behavior, guarded or non-exploitable behavior, and incomplete environment modeling. Because all tools use the same standardized oracles, their false positive rates are comparable for most vulnerability types.

Coverage. We measure coverage as the percentage of executed instructions. Figures 7 and 8 show results on small (<800 LoC)

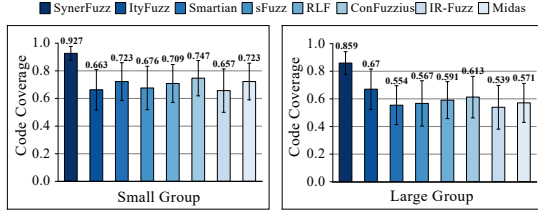


Figure 7: Code coverage achieved by different approaches across small and large contract size on benchmark B1.

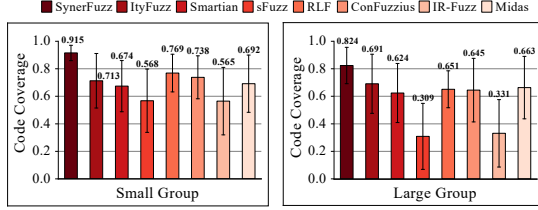


Figure 8: Code coverage achieved by different approaches across small and large contract size on benchmark B2.

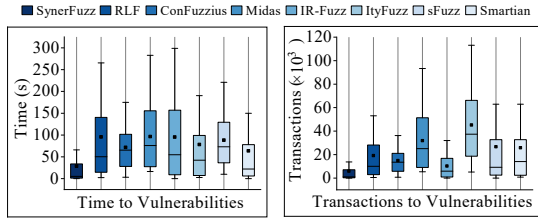


Figure 9: Time and transactions required by different approaches to detect each vulnerability in benchmark B1.

and large contracts in B1 and B2. SYNERFUZZ achieves the highest coverage for both small and large contracts, outperforming CONFUZZIUS, RLF, and ITRYFUZZ. While prior tools benefit from symbolic execution, coverage rewards, or snapshot-based state management, SYNERFUZZ consistently performs better across all settings due to its multi-dimensional guidance. This size-stratified result also reflects SYNERFUZZ’s scalability: larger contracts usually introduce more functions and inter-contract interactions, which enlarge the transaction-sequence and external-state search space, while SYNERFUZZ’s DQN policies and joint decision rule remain fixed-size and add only constant per-action inference overhead.

Answer to RQ1: Due to the introduction of multi-dimensional testing objectives, SYNERFUZZ demonstrates superior effectiveness by detecting up to 4.2× more vulnerabilities than SOTA fuzzers with the lowest false negative rates. It also achieves the highest code coverage.

5.3 RQ2: Efficiency

In Figure 9 and 10, we measure how long and how many transactions these tools need to uncover each vulnerability on benchmarks B1 and B2. SYNERFUZZ detects a vulnerability in an average of 28.6 seconds using 5,915 transactions for B1, and 85.6 seconds using 17,723 transactions for B2, reducing time and transaction overhead

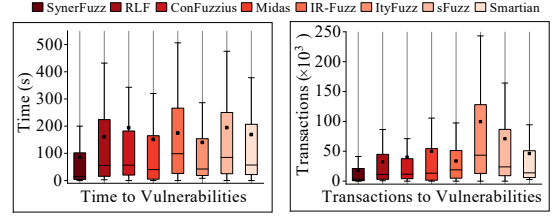


Figure 10: Time and transactions required by different approaches to detect each vulnerability in benchmark B2.

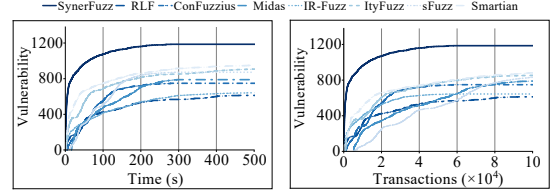


Figure 11: Cumulative vulnerability detection progress over time and transactions in benchmark B1.

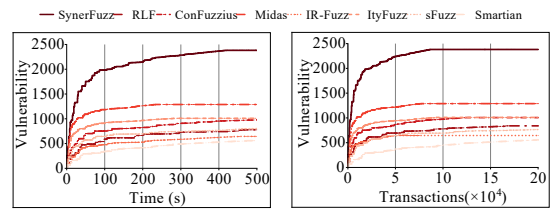


Figure 12: Cumulative vulnerability detection progress over time and transactions in benchmark B2.

by 70.6%, 86.8%, 56.5%, and 82.1% compared to SOTA tools. Additionally, SYNERFUZZ exhibits the lowest median values and variance across both metrics, demonstrating consistent efficiency across different benchmarks. Figure 11 and Figure 12 show the cumulative number of vulnerabilities that can be detected by each tool as time progresses and transaction count increases. Across both benchmarks, SYNERFUZZ detects the most vulnerabilities in the shortest time and with the fewest transactions. Notably, over 800 vulnerabilities in B1 can be detected by SYNERFUZZ within 18 seconds and 3,500 transactions, while the best performing comparative tool in Figure 11 (ITRYFUZZ) reaches only 500 within the same time. In B2, over 1,500 vulnerabilities can be detected by SYNERFUZZ within 30 seconds and 15,000 transactions, whereas the best competitor in Figure 12 (MIDAS) reaches only 1000.

Answer to RQ2: SYNERFUZZ achieves superior vulnerability detection efficiency, uncovering the most true positives while reducing time and transaction overhead by 70.6% and 86.8% compared to SOTA tools.

5.4 RQ3: DRL Contribution

We evaluate the effectiveness of our multi-agent DRL framework from two perspectives. In the training phase, figure 13 shows that all agents achieve increasing and convergent rewards over 200

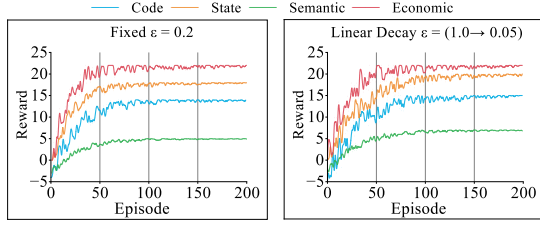


Figure 13: Training Rewards of DRL Agents under Fixed and Decaying ϵ -Greedy Strategies.

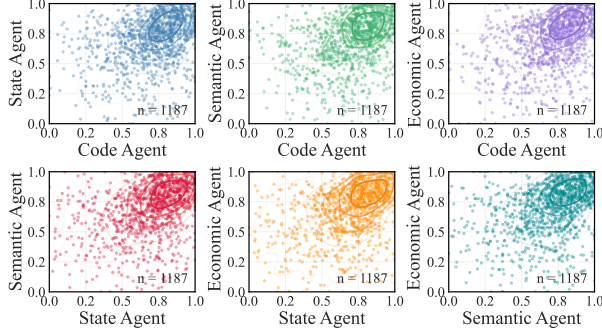


Figure 14: Normalized rewards upon vulnerability detection.

episodes. Decaying ϵ yields higher final rewards, indicating improved policy refinement. These results confirm that the DRL agents are not acting randomly but are learning to optimize long-term rewards. In the testing phase, figure 14 shows that vulnerability detection typically occurs when multiple agents achieve high rewards, with occasional complementary behaviors across agents. The economic agent is strongly correlated with others, suggesting its role in coordinating decisions. These results demonstrate that through collaboration and complementary perspectives, the learned strategies effectively guide the fuzzing process, accounting for SYNERFUZZ’s superior performance over existing tools.

Answer to RQ3: The DRL agents have learned meaningful and effective strategies that directly contribute to the improved vulnerability detection performance of SYNERFUZZ.

5.5 RQ4: Ablation Study

Fuzzing objective and agent-architecture. We conduct objective-level and architecture-level ablations on B1. For objective ablation, we remove one agent at a time: code (M_{Cd}), state (M_{Sta}), semantic (M_{Sem}), and economic (M_{Eco}). For architecture ablation, we add M_{Com} , a single-agent DQN that observes all four objective spaces and optimizes a scalarized composite reward. As shown in Table 7, removing any objective degrades performance, confirming their complementary roles. Figure 15 further tracks code, state, semantic distances, and coverage on one representative B1 case, showing that removing an agent slows progress along its corresponding objective dimension. M_{Com} also performs substantially worse than SYNERFUZZ, detecting 35.0% fewer vulnerabilities despite using all four reward signals. This shows that richer rewards alone are insufficient; SYNERFUZZ benefits from learning objective-specific policies and coordinating them during action selection.

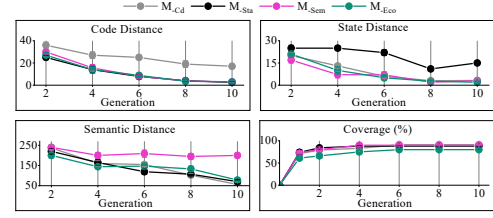


Figure 15: Evolution of metrics in contract 0x23a...deb4.

Table 7: Results of ablation configurations on benchmark B1.

	Arch.	Agent System				VulsNum		AvgTime (s)		AvgCov (%)	
		Cd.	Sta.	Sem.	Eco.	#	Impr.	#	Impr.	#	Impr.
M_{Com}	1-agent	✓	✓	✓	✓	771	-35.0%	41.0	-47.5%	75.2	-18.9%
M_{Cd}	3-agent		✓	✓	✓	985	-17.0%	43.2	-55.4%	80.8	-12.8%
M_{Sta}	3-agent	✓		✓	✓	973	-18.0%	39.8	-43.2%	78.8	-15.0%
M_{Sem}	3-agent	✓	✓		✓	938	-20.9%	34.9	-25.5%	82.3	-11.2%
M_{Eco}	3-agent	✓	✓	✓		1022	-13.9%	33.5	-20.5%	81.3	-12.35%
SYNERFUZZ	4-agent	✓	✓	✓	✓	1,187	-	27.8	-	92.7	-

Table 8: Results of agent joint-decision ablations.

	Arbitration Rule	VulsNum		AvgTime (s)		AvgCov (%)	
		#	Impr.	#	Impr.	#	Impr.
M_{Vote}	Majority voting	1,014	-14.6%	34.2	-23.0%	87.2	-5.9%
M_{PQ}	Stagnation-based priority queue	968	-18.5%	37.9	-36.3%	84.8	-8.5%
M_{ScaQ}	Scalarized normalized Q-score	1,063	-10.4%	31.3	-12.6%	88.9	-4.1%
SYNERFUZZ	Confidence-weighted preference	1,187	-	27.8	-	92.7	-

Table 9: Sensitivity analysis on one B1 validation fold.

Factor	Alternatives	Default	Δ Recall (%)	Δ Cov. (%)
Closest blocks (k)	3, 7, 10	5	[-1.6, -0.7]	[-1.0, +0.5]
Distance-reward clip (c)	3, 7, 10	5	[-1.3, -0.6]	[-0.8, -0.4]
State-target bonus (α)	0.1, 0.3, 0.4	0.2	[-1.1, +0.8]	[-0.7, +0.5]
Economic reward weights	1:2, 1:1, 4:1	2:1	[-2.1, +0.6]	[-0.5, +0.2]
DQN hidden layers	128-64, 64-64-64, 256-128-64	128-64-64	[-2.6, +0.9]	[-1.6, +1.0]

Joint-decision. We evaluate the joint decision function by keeping the four trained agents fixed and replacing only the execution-time arbitration rule. The alternatives include M_{Vote} with top-1 majority voting, M_{PQ} with priority to the least-progressing agent, and M_{ScaQ} with a fixed-weight sum of normalized Q-scores. Table 8 shows that all variants underperform SYNERFUZZ, as they discard preference strength, ignore multi-agent contributions, or rely on static weights. By contrast, SYNERFUZZ uses confidence-weighted soft arbitration, allowing all agents to contribute according to action-level certainty.

Hyperparameter sensitivity. We evaluate the sensitivity of SYNERFUZZ to five factors: closest-block count, distance-reward clip, state-target bonus scale, economic reward weight, and DQN configuration. On one held-out B1 validation fold, we vary one factor at a time and report changes in recall and coverage relative to the default setting, which achieves 84.1% recall and 92.6% coverage. As shown in Table 9, all variants cause minor changes, suggesting that SYNERFUZZ is reasonably robust to these hyperparameter and architecture choices.

Table 10: Results on historically exploited DeFi projects (B3).

Chain	TP	VERITE		TP	A1		TP	SYNERFUZZ	
		FN	Profit		FN	Profit		FN	Profit
ETH	4	N/A	–	9	2	–	9	2	–
BSC	23	N/A	–	17	8	–	24	1	–
Total	27	N/A	\$18.3M	26	10	\$9.3M	33	3	\$22.1M

Table 11: 0-day vulnerabilities found by SYNERFUZZ.

ID	Location	Severity	Feedback	Program	TVL
1	withdrawForwardFee()	Critical	Fixed	zkLink Nova	\$15.26 M
2	WithdrawalMessage()	Medium	Fixed		
3	deposit()	Medium	Fixed	Lumoz	\$537.38 K
4	deposit()	Medium	Fixed		
5	verifySignatures()	Medium	ACK		
6	withdraw()	Medium	Fixed		
7	_deposit()	Medium	ACK		
8	claim()	Low	ACK		
9	claim()	Low	Fixed		
10	bridgeAsset()	Low	ACK		
11	distributeRewardUsdc()	Medium	Fixed	Gambit-Wemix	\$154 M
12	withdraw()	Critical	Fixed	GoMutual	Unknown
13	Comptroller.sol	Low	Fixed		
14	Comptroller.sol	Low	Fixed		
15	Comptroller.sol	Low	ACK		
16	SurplusPool.sol	Low	ACK		
17	SurplusPool.sol	Low	ACK		

Answer to RQ4: Each dimension contributes to SYNERFUZZ, and both the composite-agent and arbitration ablations confirm the need for objective-specific learning and confidence-weighted coordination.

5.6 RQ5: Real-World Evaluation

Overall results. We evaluate SYNERFUZZ on 36 DeFi projects in B3 that had previously been exploited to extract profit. Since VERITE and A1 are not open-sourced, we compare against their reported experimental results. For each project, SYNERFUZZ forks the pre-exploit on-chain state and initializes an attacker account with a seed balance. It then fuzzes transaction sequences while monitoring value flows and balance changes to detect profit-generating behaviors. As shown in Table 10, SYNERFUZZ reproduces 33 out of 36 exploits and recovers \$22.1M profit, outperforming prior approaches in detecting profitable vulnerabilities. Among the three missed historical exploits, one requires adversarial block-timing manipulation for oracle-state changes, which is not modeled in the current observation space. The other two require deep multi-protocol composition across nested call chains, exceeding the scope of our current semantic/state objective extraction.

We then evaluated SYNERFUZZ on *Secure3* bug bounty programs and revealed 25 previously unknown vulnerabilities. Of these, 17 were subsequently confirmed or remediated by project teams, resulting in a \$3.8K USD bounty award. As detailed in Table 11, the validated vulnerabilities are distributed across four projects and include two critical-risk and seven medium-risk issues. Collectively, these vulnerabilities potentially impact protocols managing over \$179.8 million in total value locked.

Case Study. SYNERFUZZ identified a reentrancy in GoMutual’s ERC20AssetGateway contract, as shown in Listing 2. SYNERFUZZ first matched a known reentrancy pattern from its semantic library and extracted the associated critical state conditions. By setting these as testing objectives in the semantic and state dimensions,

```

1 IERC20(tokenAddress).transferFrom(from, address(this), amount)
2 address originPayer = depositionCertificates[planId].depositor
3 function withdraw(uint256 planId) public returns (uint256) {
4     certificate = depositionCertificates[planId];
5     address payer = certificate.depositor;
6     require(payer == msg.sender, "EAG_04");
7     require(certificate.amount > 0, "EAG_03");
8     uint256 amount = certificate.amount;
9     IERC20(...).transfer(certificate.depositor, amount);
10    certificate.amount = 0; // ...

```

Listing 2: Snippet of GoMutual with Reentrancy vulnerability

SYNERFUZZ successfully triggered the vulnerable reentrancy via external token transfers before updating internal state. In contrast, existing fuzzers struggle to construct such coordinated transaction sequences due to the lack of multi-dimensional awareness and targeted objective-driven guidance.

Answer to RQ5: SYNERFUZZ shows real-world impact by reproducing 33 profitable historical exploits and identifying 17 0-day vulnerabilities, resulting in \$3.8K awards and securing \$179.8 million USD.

6 Related Work

Smart contract fuzzing. CONTRACTFUZZER [22], the first contract fuzzer, generates inputs based on ABI. Subsequent tools have introduced various improvements. sFUZZ [34] adapts AFL’s architecture with coverage-guided strategies; HARVEY [48] employs branch distance prediction to generate inputs likely to cover new paths; Recent advancements include IryFUZZ [38], which identifies state-rich contracts through data flow path points.

DRL has emerged as a powerful paradigm for solving decision-making problems [12, 21, 27]. DRL enables agents to learn optimal policies through environmental interactions without explicit programming. Its effectiveness has been demonstrated in intelligent transportation systems [24], robotics [41], natural language processing [4], and other fields [5, 28, 37]. In SYNERFUZZ, we leverage DRL to learn optimal fuzzing strategies across multiple dimensions, enhancing the effectiveness of contract vulnerability detection.

7 Conclusion

In this paper, we introduce SYNERFUZZ, a smart contract fuzzer based on multi-dimensional objective learning. We formulate fuzzing as a Dec-POMDP framework with DRL agents to learn effective fuzzing strategies. Evaluations show SYNERFUZZ’s superior effectiveness in curated datasets, and strong real-world impact by reproducing 33 historical exploits and identifying 17 0-day vulnerabilities.

Acknowledgement

This research was supported in part by the National Natural Science Foundation of China under Grants No. 62302343 and No. 62472323, the Key R&D Program of Hubei Province under Grant No. 2024BAB018, the Wuhan Scientific and Technical Achievements Project under Grant No. 2024030803010172, and the National Cryptologic Science Fund of China under Grant No. 2025NCSF02027.

Data Availability

The artifacts is available at: <https://doi.org/10.6084/m9.figshare.32939534>, including the codebase and datasets.

References

- [1] Alchemy. 2024. Alchemy. Retrieved April 20, 2026 from <https://www.alchemy.com/supernode>
- [2] Cornelius Aschermann, Sergej Schumilo, Ali Abbasi, and Thorsten Holz. 2020. Ijon: Exploring Deep State Spaces via Fuzzing. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. doi:10.1109/SP40000.2020.00117
- [3] Kushal Babel, Philip Daian, Mahimna Kelkar, and Ari Juels. 2023. Clockwork Finance: Automated Analysis of Economic Security in Smart Contracts. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. doi:10.1109/SP46215.2023.10179346
- [4] Pallavi Bagga, Nicola Paoletti, and Kostas Stathis. 2025. Adaptive Strategy Templates Using Deep Reinforcement Learning for Multi-Issue Bilateral Negotiation. *Neurocomputing* 623 (2025). doi:10.1016/j.neucom.2025.129381
- [5] Zheng Bao, Changbing Tang, Xinghuo Yu, Feilong Lin, Guanghui Wen, and Zhonglong Zheng. 2025. Cost-Effective Power Delivery via Deep Reinforcement Learning-Based Dynamic Electric Vehicle Transportation. *IEEE Internet of Things Journal* (2025). doi:10.1109/JIOT.2025.3552823
- [6] Daniel S. Bernstein, Robert Givan, Neil Immerman, and Shlomo Zilberstein. 2002. The Complexity of Decentralized Control of Markov Decision Processes. *Mathematics of Operations Research* 27 (2002). doi:10.1287/moor.27.4.819.297
- [7] Chong Chen, Jianzhong Su, Jiachi Chen, Yanlin Wang, Tingting Bi, Jianxing Yu, Yanli Wang, Xingwei Lin, Ting Chen, and Zibin Zheng. 2025. When ChatGPT Meets Smart Contract Vulnerability Detection: How Far Are We? *ACM Transactions on Software Engineering and Methodology* 34 (2025). doi:10.1145/3702973
- [8] Jaeseung Choi, Doyeon Kim, Soomin Kim, Gustavo Grieco, Alex Groce, and Sang Kil Cha. 2021. SMARTIAN: Enhancing Smart Contract Fuzzing with Static and Dynamic Data-Flow Analyses. In *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*. doi:10.1109/ASE51524.2021.9678888
- [9] Code4rena. 2026. Code4rena Audit Reports. Retrieved June 22, 2026 from <https://code4rena.com/reports>
- [10] DeFiHackLabs. 2025. DeFiHackLabs. Retrieved January 20, 2026 from <https://defihacklabs.io/>
- [11] Monika di Angelo and Gernot Salzer. 2023. Consolidation of Ground Truth Sets for Weakness Detection in Smart Contracts. In *Proceedings of the International Conference on Financial Cryptography and Data Security Workshops (FC)*. doi:10.1007/978-3-031-48806-1_28
- [12] Linkang Du, Min Chen, Mingyang Sun, Shouling Ji, Peng Cheng, Jiming Chen, and Zhikun Zhang. 2024. ORL-Auditor: Dataset Auditing in Offline Deep Reinforcement Learning. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. doi:10.14722/ndss.2024.23184
- [13] Thomas Durieux, João F. Ferreira, Rui Abreu, and Pedro Cruz. 2020. Empirical Review of Automated Analysis Tools on 47,587 Ethereum Smart Contracts. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*. doi:10.1145/3377811.3380364
- [14] Ethereum. 2023. py-evm. Retrieved March 15, 2026 from <https://github.com/ethereum/py-evm>
- [15] Farama-Foundation. 2022. PettingZoo. Retrieved March 27, 2026 from <https://pettingzoo.farama.org>
- [16] Josselin Feist, Gustavo Grieco, and Alex Groce. 2019. Slither: A Static Analysis Framework for Smart Contracts. In *Proceedings of the IEEE/ACM International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. doi:10.1109/WETSEB.2019.00008
- [17] Christof Ferreira Torres, Antonio Ken Iannillo, Arthur Gervais, and Radu State. 2021. ConFuzzius: A Data Dependency-Aware Hybrid Fuzzer for Smart Contracts. In *Proceedings of the IEEE European Symposium on Security and Privacy (EuroS&P)*. doi:10.1109/EuroSP51992.2021.00018
- [18] Foundry. 2026. A Fast, Portable, and Modular Toolkit for Ethereum Application Development Written in Rust. Retrieved July 5, 2026 from <https://www.getfoundry.sh/>
- [19] Hongyan Gao, Yibiao Yang, Maolin Sun, Jiangchang Wu, Yuming Zhou, and Baowen Xu. 2025. ClozeMaster: Fuzzing Rust Compiler by Harnessing LLMs for Infilling Masked Real Programs. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*. doi:10.1109/ICSE55347.2025.00175
- [20] Arthur Gervais and Liyi Zhou. 2025. AI Agent Smart Contract Exploit Generation. *arXiv preprint arXiv:2507.05558* (2025). doi:10.48550/arXiv.2507.05558
- [21] Soumyadeep Hore, Jalal Ghadermazi, Diwas Paudel, Ankit Shah, Tapas Das, and Nathaniel Bastian. 2025. Deep PackGen: A Deep Reinforcement Learning Framework for Adversarial Network Packet Generation. *ACM Transactions on Privacy and Security* 28 (2025). doi:10.1145/3712307
- [22] Bo Jiang, Ye Liu, and W. K. Chan. 2018. ContractFuzzer: Fuzzing Smart Contracts for Vulnerability Detection. In *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*. doi:10.1145/3238147.3238177
- [23] Ziqiao Kong, Cen Zhang, Maoyi Xie, Ming Hu, Yue Xue, Ye Liu, Haijun Wang, and Yang Liu. 2025. Smart Contract Fuzzing Towards Profitable Vulnerabilities. In *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*. doi:10.1145/3715720
- [24] Mengge Li, Miao Ma, Liang Wang, Zhao Pei, Jie Ren, and Bo Yang. 2024. Multiagent Deep Reinforcement Learning Based Incentive Mechanism for Mobile Crowdsensing in Intelligent Transportation Systems. *IEEE Systems Journal* 18 (2024). doi:10.1109/JSYST.2024.3351310
- [25] Zhaoxuan Li, Ziming Zhao, Wenhao Li, Rui Zhang, Rui Xue, Siqu Lu, and Fan Zhang. 2024. Demo: Enhancing Smart Contract Security Comprehensively through Dynamic Symbolic Execution. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. doi:10.1145/3658644.3691365
- [26] Jiayi Lin, Qingyu Zhang, Junzhe Li, Chenxin Sun, Hao Zhou, Changhua Luo, and Chenxiong Qian. 2025. Automatic Library Fuzzing through API Relation Evolution. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. doi:10.14722/ndss.2025.240750
- [27] Xin Lin, Aijun Liu, Chen Han, Xiaohu Liang, Yifu Sun, Guoru Ding, and Haibo Zhou. 2025. Intelligent Adaptive MIMO Transmission for Nonstationary Communication Environment: A Deep Reinforcement Learning Approach. *IEEE Transactions on Communications* (2025). doi:10.1109/TCOMM.2025.3529263
- [28] Yicen Liu and Junning Zhang. 2024. Service Function Chain Embedding Meets Machine Learning: Deep Reinforcement Learning Approach. *IEEE Transactions on Network and Service Management* 21 (2024). doi:10.1109/TNSM.2024.3353808
- [29] Zhenguang Liu, Peng Qian, Jiaxu Yang, Lingfeng Liu, Xiaojun Xu, Qinning He, and Xiaosong Zhang. 2023. Rethinking Smart Contract Fuzzing: Fuzzing With Invocation Ordering and Important Branch Revisiting. *IEEE Transactions on Information Forensics and Security* (2023). doi:10.1109/TIFS.2023.3237370
- [30] Abtin Ijadi Maghsoodi and Mehdi Rajabi Asadabadi. 2025. A Stratified Markovian Multipreference Decision Support System to Assess Supply Chain Blockchain Platforms. *IEEE Transactions on Engineering Management* 72 (2025). doi:10.1109/TEM.2024.3521655
- [31] Sajad Meisami, Hugo Dabadie, Song Li, Yuzhe Tang, and Yue Duan. 2025. SigScope: Detecting and Understanding Off-Chain Message Signing-Related Vulnerabilities in Decentralized Applications. In *Proceedings of the ACM Web Conference (WWW)*. doi:10.1145/3696410.3714686
- [32] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. 2015. Human-Level Control through Deep Reinforcement Learning. *Nature* 518 (2015). doi:10.1038/nature14236
- [33] Marcus Nachtigall, Lisa Nguyen Quang Do, and Eric Bodden. 2019. Explaining Static Analysis — A Perspective. In *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*. doi:10.1109/ASEW.2019.00023
- [34] Tai D. Nguyen, Long H. Pham, Jun Sun, Yun Lin, and Quang Tran Minh. 2020. sFuzz: An Efficient Adaptive Fuzzer for Solidity Smart Contracts. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*. doi:10.1145/3377811.3380334
- [35] Rohan Padhye, Caroline Lemieux, Koushik Sen, Laurent Simon, and Hayawardh Vijayakumar. 2019. FuzzFactory: Domain-Specific Fuzzing with Waypoints. In *Proceedings of the ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. doi:10.1145/3360600
- [36] Secure3. 2022. Comprehensive Security Audits for the Web3 Ecosystems. Retrieved May 23, 2025 from <https://app.secure3.io>
- [37] Jaemin Seo, SangKyeun Kim, Azarakhsh Jalalvand, Rory Conlin, Andrew Rothstein, Joseph Abbate, Keith Erickson, Josiah Wai, Ricardo Shousha, and Egemen Kolenen. 2024. Avoiding Fusion Plasma Tearing Instability with Deep Reinforcement Learning. *Nature* 626 (2024). doi:10.1038/s41586-024-07024-9
- [38] Chaofan Shou, Shangyin Tan, and Koushik Sen. 2023. ItyFuzz: Snapshot-Based Fuzzer for Smart Contract. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. doi:10.1145/3597926.3598059
- [39] Sunbeom So, Myungho Lee, Jisu Park, Heejo Lee, and Hakjoo Oh. 2020. VeriSmart: A Highly Precise Safety Verifier for Ethereum Smart Contracts. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. doi:10.1109/SP40000.2020.00032
- [40] Jianzhong Su, Hong-Ning Dai, Lingjun Zhao, Zibin Zheng, and Xiapu Luo. 2023. Effectively Generating Vulnerable Transaction Sequences in Smart Contracts with Reinforcement Learning-Guided Fuzzing. In *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*. doi:10.1145/3551349.3560429
- [41] Chen Tang, Ben Abbatematteo, Jiaheng Hu, Rohan Chandra, Roberto Martin-Martin, and Peter Stone. 2025. Deep Reinforcement Learning for Robotics: A Survey of Real-World Successes. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. doi:10.1609/aaai.v39i27.35095
- [42] Thu-ml. 2020. Tianshou. Retrieved March 10, 2026 from <https://tianshou.org/en/stable/>
- [43] Trail of Bits. 2026. Trail of Bits Publications and Audit Reports. Retrieved June 22, 2026 from <https://blog.trailofbits.com/categories/audits/>
- [44] Fernando Richter Vidal, Naghme Ivaki, and Nuno Laranjeiro. 2026. An Open Hierarchical Taxonomy for Smart Contract Vulnerabilities. Retrieved June 30, 2026 from <https://opensvc.dei.uc.pt/>

- [45] Jinghan Wang, Yue Duan, Wei Song, Heng Yin, and Chengyu Song. 2019. Be Sensitive and Collaborative: Analyzing Impact of Coverage Metrics in Greybox Fuzzing. In *Proceedings of the International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*.
- [46] Gavin Wood et al. 2014. Ethereum: A Secure Decentralised Generalised Transaction Ledger. *Ethereum Project Yellow Paper* 151 (2014).
- [47] Shuohan Wu, Zihao Li, Luyi Yan, Weimin Chen, Muhui Jiang, Chenxu Wang, Xiapu Luo, and Hao Zhou. 2024. Are We There Yet? Unraveling the State-of-the-Art Smart Contract Fuzzers. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*. doi:10.1145/3597503.3639152
- [48] Valentin Wüstholz and Maria Christakis. 2020. Harvey: A Greybox Fuzzer for Smart Contracts. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. doi:10.1145/3368089.3417064
- [49] Chenyuan Yang, Zijie Zhao, and Lingming Zhang. 2025. KernelGPT: Enhanced Kernel Fuzzing via Large Language Models. In *Proceedings of the ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. doi:10.1145/3676641.3716022
- [50] Chavhan Sujeet Yashavant, Saurabh Kumar, and Amey Karkare. 2022. ScrawlID: A Dataset of Real World Ethereum Smart Contracts Labelled with Vulnerabilities. *arXiv preprint arXiv:2202.11409* (2022). doi:10.48550/arXiv.2202.11409
- [51] Mingxi Ye, Xingwei Lin, Yuhong Nan, Jiajing Wu, and Zibin Zheng. 2024. Midas: Mining Profitable Exploits in On-Chain Smart Contracts via Feedback-Driven Fuzzing and Differential Analysis. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. doi:10.1145/3650212.3680321
- [52] Andreas Zeller, Rahul Gopinath, Marcel Böhme, Gordon Fraser, and Christian Holler. 2020. The Fuzzing Book. Retrieved May 12, 2025 from <https://www.fuzzingbook.org/>
- [53] Zhuo Zhang, Brian Zhang, Wen Xu, and Zhiqiang Lin. 2023. Demystifying Exploitable Bugs in Smart Contracts. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*. doi:10.1109/ICSE48619.2023.00061